

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Utilização de bibliotecas de Ciência de Dados em Python

Introdução às bibliotecas de Ciência de Dados em Python

Introdução às bibliotecas de Ciência de Dados
em Python

Código da aula: [DADOS]ANO1C1B4S32A1

Utilização de
bibliotecas de
Ciência de Dados
em Python

Mapa da unidade 7 Componente 1

semana

32

Você está aqui!

Introdução às bibliotecas de
Ciência de Dados em Python

**Utilização de
bibliotecas de
Ciência de Dados
em Python**

Mapa da unidade 7 Componente 1

Você está aqui!

Introdução às bibliotecas de
Ciência de Dados em Python

Aula 1

Código da aula: [DADOS]ANO1C1B4S32A1

32



Objetivos da aula

- Introduzir e utilizar as principais bibliotecas de Ciência de Dados em Python.



Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou à internet;
- Software Anaconda/Jupyter Notebook instalado ou similar.



Duração da aula

50 minutos.



Competências técnicas

- Usar ferramentas de desenvolvimento de software como o Jupyter Notebook;
- Compreender e dominar as principais bibliotecas de Ciência de Dados em Python.



Competências socioemocionais

- Analisar informações de forma crítica e tomar decisões;
- Avaliar a qualidade dos dados e a eficácia dos modelos e das técnicas de análise;
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados;
- Trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes.



© Getty Images

Primeiras ideias

O que você entende por bibliotecas de Ciência de Dados em Python?

Você acha que essas bibliotecas podem ser úteis em um projeto de Ciência de Dados?

Quais são algumas das bibliotecas populares de Ciência de Dados em Python?

Ponto de partida

Situação:

Imagine que você é um analista de dados em uma empresa de tecnologia e recebeu uma grande quantidade de dados de vendas de produtos nos últimos cinco anos. Seu gerente pediu para você preparar um relatório que mostre:

1. As tendências de vendas ao longo do tempo.
2. Os produtos mais vendidos em cada ano.
3. Uma análise visual dos dados para identificar padrões sazonais.

Discussão:

1. Quais seriam os primeiros passos para começar a analisar esses dados?
2. Quais tipos de gráficos ou visualizações você acha que seriam úteis para representar os dados de vendas?

Construindo
o **conceito**

O que são bibliotecas de Ciência de Dados?

Bibliotecas de Ciência de Dados são coleções de módulos e funções em Python que ajudam a realizar tarefas de análise, manipulação e visualização de dados, além de aprendizado de máquina.

Elas são projetadas para simplificar o trabalho dos cientistas de dados, permitindo que eles se concentrem nas análises em vez de desenvolver algoritmos complexos do zero.

Construindo
o **conceito**

Importância das bibliotecas de Ciência de Dados

As bibliotecas de Ciência de Dados são essenciais porque fornecem ferramentas padronizadas para trabalhar com dados.

Elas ajudam a economizar tempo, aumentam a eficiência e garantem que os cientistas de dados possam aplicar métodos estatísticos e de aprendizado de máquina de forma consistente.

Como seria se elas não existissem?

Sem essas bibliotecas, seria necessário implementar manualmente muitos dos algoritmos e das técnicas usados regularmente, o que seria muito trabalhoso e propenso a erros.

Construindo o **conceito**

Vamos conhecer na prática!

Como instalar as bibliotecas:

A instalação das bibliotecas de Ciência de Dados é um processo simples, realizado por meio do gerenciador de **pacotes pip**. Lembre-se de seguir as instruções do professor para acessar o Jupyter via Anaconda.

Aqui estão os comandos para instalar as principais bibliotecas discutidas:

Biblioteca	Comando
NumPy	<code>pip install numpy</code>
Pandas	<code>pip install pandas</code>
Matplotlib	<code>pip install matplotlib</code>
Seaborn	<code>pip install seaborn</code>

Construindo o **conceito**

NumPy

NumPy (*Numerical Python*) é uma biblioteca fundamental para computação científica em Python.

Ela fornece suporte para arrays e matrizes de grandes dimensões, junto com uma coleção de funções matemáticas para operar esses arrays.



Tome nota

NumPy é usado como base para muitas outras bibliotecas de Ciência de Dados devido à sua eficiência e flexibilidade.

Construindo o conceito

NumPy: vamos conhecer na prática!

Exemplo 1: transformações matemáticas em arrays

Vamos criar um exemplo em que aplicamos várias transformações matemáticas em arrays utilizando NumPy.

Suponha que temos um conjunto de dados representando a temperatura diária de uma cidade durante um mês e queremos realizar algumas análises estatísticas e transformações.

```
import numpy as np

# Dados de temperatura (em graus Celsius) para um mês (30 dias)
temperatures = np.array([15.5, 16.0, 15.2, 14.8, 15.0, 15.3, 15.8, 16.5, 17.0, 17.2,
                          16.8, 16.4, 16.1, 15.7, 15.4, 15.0, 14.7, 14.4, 14.1, 14.0,
                          14.5, 15.0, 15.5, 16.0, 16.5, 17.0, 17.5, 18.0, 18.5, 19.0])

# Calculando a média e o desvio padrão
mean_temp = np.mean(temperatures)
std_temp = np.std(temperatures)
print(f"Média da temperatura: {mean_temp:.2f} °C")
print(f"Desvio padrão da temperatura: {std_temp:.2f} °C")

# Normalizando os dados (Z-score)
normalized_temps = (temperatures - mean_temp) / std_temp
print("Temperaturas normalizadas (Z-score):")
print(normalized_temps)

# Convertendo as temperaturas de Celsius para Fahrenheit
temps_fahrenheit = (temperatures * 9/5) + 32
print("Temperaturas em Fahrenheit:")
print(temps_fahrenheit)

# Encontrando os dias com temperatura acima da média
above_avg_days = np.where(temperatures > mean_temp)
print("Dias com temperatura acima da média:")
print(above_avg_days[0] + 1) # Adicionando 1 para representar o dia corretamente
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo
o **conceito**

Pandas

Pandas é uma biblioteca de manipulação e análise de dados que oferece estruturas rápidas, flexíveis e expressivas, projetadas para facilitar o trabalho com dados relacionais ou rotulados.

A principal estrutura do Pandas é o DataFrame, que permite a manipulação de dados tabulares de maneira intuitiva e eficiente.

Construindo o conceito

Pandas: vamos conhecer na prática!

Exemplo 2: análise de dados financeiros

Vamos criar um exemplo em que analisamos um conjunto de dados financeiros usando o Pandas.

Suponha que temos dados diários do preço das ações de uma empresa e queremos calcular o retorno diário, a média móvel de 7 dias e identificar os dias em que o preço fechou acima da média móvel.

```
import pandas as pd

# Dados de preços das ações para 30 dias
data = {
    'Date': pd.date_range(start='2023-01-01', periods=30),
    'Close': [150, 152, 153, 155, 154, 156, 158, 157, 159, 160, 161, 162, 164, 165,
             166, 168, 170, 172, 171, 173, 175, 174, 176, 178, 177, 179, 180, 181,
             183, 185]
}
df = pd.DataFrame(data)

# Calculando o retorno diário
df['Daily Return'] = df['Close'].pct_change()
print("Retorno diário:")
print(df[['Date', 'Daily Return']].head())

# Calculando a média móvel de 7 dias
df['7-Day MA'] = df['Close'].rolling(window=7).mean()
print("Média móvel de 7 dias:")
print(df[['Date', 'Close', '7-Day MA']].head(10))

# Identificando os dias em que o preço fechou acima da média móvel
df['Above 7-Day MA'] = df['Close'] > df['7-Day MA']
print("Dias com fechamento acima da média móvel de 7 dias:")
print(df[['Date', 'Close', '7-Day MA', 'Above 7-Day MA']].head(15))
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o **conceito**

Matplotlib

Matplotlib é a **principal biblioteca de plotagem 2D em Python**, permitindo a criação de visualizações estáticas, animadas e interativas.

É bastante utilizado para criar gráficos de linhas, gráficos de barras, histogramas e muitos outros tipos de visualizações. Com isso, Matplotlib é altamente personalizável, permitindo ajustes detalhados nas figuras geradas.



Dica

Com Matplotlib, é possível alterar cores, tipos de linha, tamanhos de fonte e muito mais para criar gráficos que atendam exatamente às necessidades do usuário.

Construindo o conceito

Matplotlib: vamos conhecer na prática!

Exemplo 3: visualização de vendas mensais

Vamos criar um exemplo em que visualizamos os dados de vendas mensais de um produto ao longo de um ano usando Matplotlib.

Vamos plotar um gráfico de barras mostrando as vendas mensais e um gráfico de linha mostrando a média móvel de três meses.

```
import matplotlib.pyplot as plt

# Dados de vendas mensais (em unidades) para um ano
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
sales = [250, 300, 350, 400, 380, 420, 460, 480, 450, 430, 420, 400]

# Calculando a média móvel de 3 meses
rolling_avg = pd.Series(sales).rolling(window=3).mean()

# Criando a figura e os eixos
fig, ax = plt.subplots(figsize=(10, 6))

# Plotando o gráfico de barras para vendas mensais
ax.bar(months, sales, color='skyblue', label='Vendas Mensais')

# Plotando o gráfico de linha para a média móvel
ax.plot(months, rolling_avg, color='orange', marker='o', label='Média Móvel (3 meses)')

# Adicionando títulos e rótulos
ax.set_title('Vendas Mensais e Média Móvel')
ax.set_xlabel('Meses')
ax.set_ylabel('Unidades Vendidas')

# Adicionando legenda
ax.legend()

# Exibindo o gráfico
plt.show()
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o **conceito**

Seaborn

Seaborn é uma biblioteca de visualização de dados baseada no Matplotlib, mas oferece uma interface de nível superior para desenhar gráficos estatísticos atraentes e informativos.

Essa biblioteca facilita a criação de gráficos complexos com menos código e inclui funcionalidades integradas para lidar com conjuntos de dados complexos.



Dica

Com Seaborn, é possível criar gráficos de dispersão, gráficos de violino, de caixa, de correlação, entre outros, de forma simples e rápida.

Construindo o conceito

Seaborn: vamos conhecer na prática!

Exemplo 4: análise de distribuição de gorjetas

Vamos criar um exemplo em que analisamos a distribuição de gorjetas em um restaurante usando Seaborn.

Vamos usar o conjunto de dados de gorjetas do Seaborn para criar gráficos de violino, gráficos de caixa e gráficos de dispersão para entender melhor a distribuição e a relação entre variáveis.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Carregando o conjunto de dados de gorjetas
tips = sns.load_dataset("tips")

# Criando um gráfico de violino para a distribuição de gorjetas por dia
plt.figure(figsize=(10, 6))
sns.violinplot(x="day", y="tip", data=tips, palette="muted")
plt.title('Distribuição de Gorjetas por Dia')
plt.xlabel('Dia da Semana')
plt.ylabel('Gorjeta ($)')
plt.show()

# Criando um gráfico de caixa para a distribuição de gorjetas por dia e por tempo de refeição
plt.figure(figsize=(10, 6))
sns.boxplot(x="day", y="tip", hue="time", data=tips, palette="coolwarm")
plt.title('Distribuição de Gorjetas por Dia e Tempo de Refeição')
plt.xlabel('Dia da Semana')
plt.ylabel('Gorjeta ($)')
plt.legend(title='Tempo de Refeição')
plt.show()

# Criando um gráfico de dispersão para a relação entre o total da conta e a gorjeta
plt.figure(figsize=(10, 6))
sns.scatterplot(x="total_bill", y="tip", hue="time", style="time", data=tips, palette="deep")
plt.title('Relação entre Total da Conta e Gorjeta')
plt.xlabel('Total da Conta ($)')
plt.ylabel('Gorjeta ($)')
plt.legend(title='Tempo de Refeição')
plt.show()
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



Vamos
fazer um
quiz

O que é NumPy?

Uma linguagem de
programação.

Uma biblioteca de
visualização de dados.

Uma biblioteca de
computação científica.

Uma ferramenta de
banco de dados.



Vamos
fazer um
quiz

Qual é a principal estrutura de dados do Pandas?

Lista.

Matriz.

DataFrame.

Dicionário.



Vamos
fazer um
quiz

**Qual biblioteca é baseada no
Matplotlib para visualização
de dados?**

Numpy.

Pandas.

Seaborn.

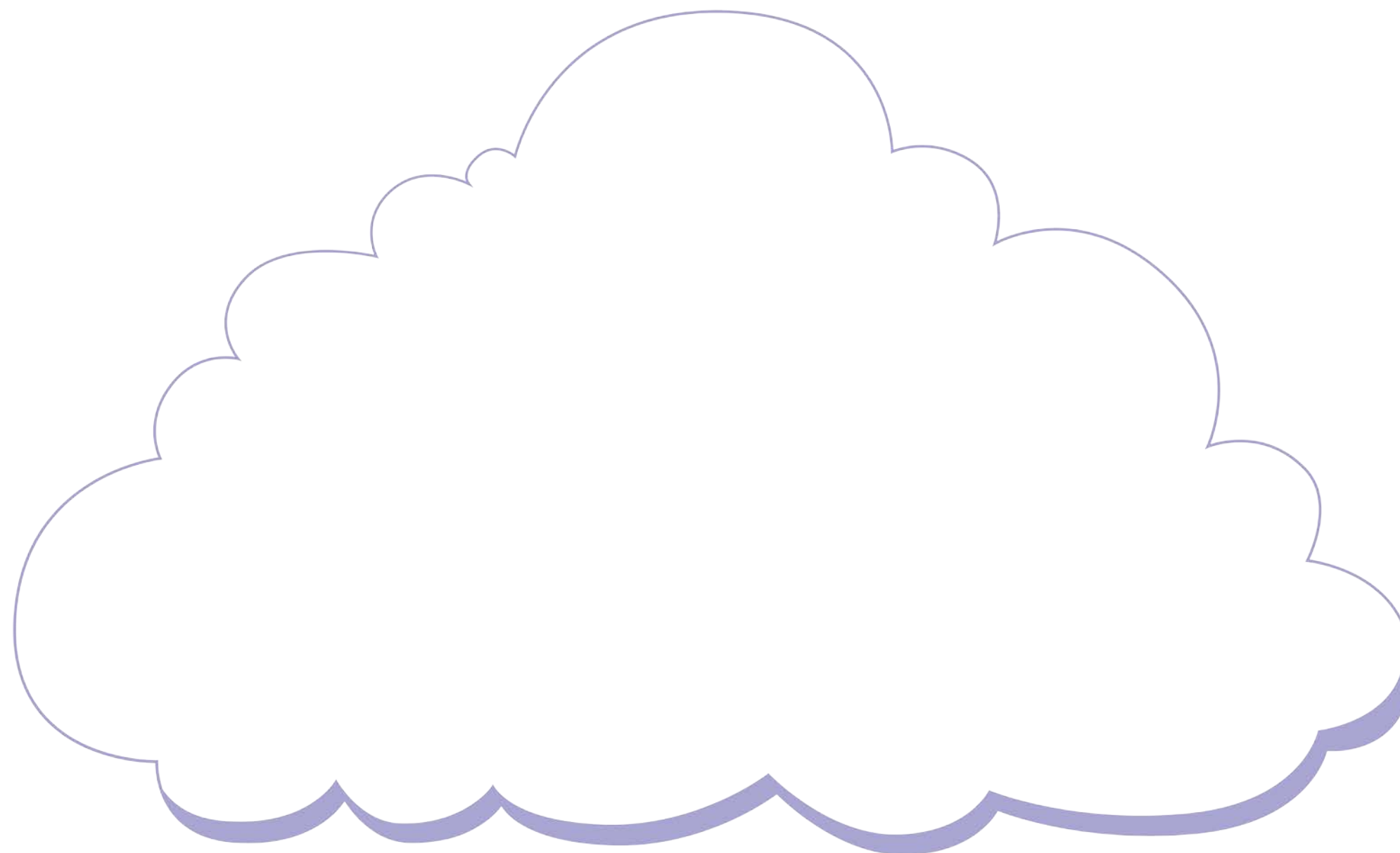
SciPy.

Nuvem de palavras



© Getty Images

O que nós
**aprendemos
hoje?**





© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** A introdução às bibliotecas de ciência de dados em Python abrange ferramentas essenciais como NumPy, Pandas, Matplotlib e Seaborn para análise, manipulação e visualização de dados.
- 2** NumPy suporta arrays e operações matemáticas eficientes, Pandas facilita a manipulação de dados tabulares e Matplotlib e Seaborn criam visualizações informativas.
- 3** Essas bibliotecas economizam tempo e garantem precisão, permitindo que cientistas de dados foquem insights e interpretação.

Saiba mais

Domine os gráficos em *data science*!

Aprenda a criar visuais impressionantes com Matplotlib, Seaborn e Plotly.

Personalize suas visualizações com facilidade e aplique as melhores práticas para destacar seus dados como nunca antes! Acesse o link abaixo e saiba como:

ALURA. *Data Visualization*: criando gráficos com bibliotecas Python. Disponível em: <https://www.alura.com.br/curso-online-data-visualization-graficos-bibliotecas-python>. Acesso em: 20 ago. 2024.

Referências da aula

Identidade visual: imagens © Getty Images

JUPYTER. *Página inicial*, [s.d.]. Disponível em: <https://jupyter.org>. Acesso em: 20 ago. 2024.

MATPLOTLIB. *Página inicial*, [s.d.]. Disponível em: <https://matplotlib.org>. Acesso em: 20 ago. 2024.

NUMPY. *Página inicial*, [s.d.]. Disponível em: <https://numpy.org>. Acesso em: 20 ago. 2024.

PANDAS. *Página inicial*, [s.d.]. Disponível em: <https://pandas.pydata.org>. Acesso em: 20 ago. 2024.

SEABORN. *Página inicial*, [s.d.]. Disponível em: <https://seaborn.pydata.org>. Acesso em: 20 ago. 2024.

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**